

Práctica 4: Ramificación y Poda, Programación Lineal

Algoritmia Básica

Imad (901421)

Bellido (903080)

Mayo 2026

1. Diseño del Algoritmo de Ramificación y Poda

El problema consiste en particionar un conjunto de N participantes en $N/3$ equipos de 3 personas, minimizando el conflicto total. Este problema es equivalente a encontrar una partición de un grafo completo ponderado en clicas de tamaño 3 con peso mínimo.

1.1. Representación de Soluciones y Árbol de Búsqueda

La solución se construye asignando participantes $0, 1, \dots, N - 1$ de forma secuencial. Un estado del árbol de búsqueda se define por:

- El participante p que se está asignando actualmente.
- La configuración actual de los equipos (quién está en cada equipo).
- El conflicto acumulado hasta el momento.

Para evitar simetrías y reducir el espacio de búsqueda, se imponen las siguientes reglas:

1. Un participante solo puede unirse a un equipo ya existente que no esté lleno (menos de 3 miembros) o iniciar el siguiente equipo vacío disponible.
2. El orden de los equipos no importa, por lo que no se permite saltar equipos vacíos.

1.2. Función de Cota Inferior (Lower Bound)

Para podar ramas no prometedoras, se utiliza una cota inferior $h(n)$ que estima el conflicto mínimo que añadirán los participantes restantes. Sea $W_{i,j} = C_{i,j} + C_{j,i}$ el conflicto mutuo entre i y j . El conflicto total G se puede expresar como $\frac{1}{2} \sum_{i=0}^{N-1}$ conflicto de i con sus dos compañeros.

La cota inferior se calcula sumando para cada participante i el conflicto mínimo con sus socios necesarios:

- Si i ya tiene 2 socios en su equipo, no añade nada a la cota futura.
- Si i tiene 1 socio j , añade al menos $W_{i,j} + \min_k W_{i,k}$.
- Si i no tiene socios asignados, añade al menos la suma de sus dos menores $W_{i,k}$.

La suma de estas estimaciones se divide por 2 para obtener el nivel de conflicto (ya que cada par $\{i, j\}$ se contaría dos veces).

2. Implementación

El algoritmo se ha implementado en C++ utilizando un enfoque de **Backtracking con Poda (DFS Branch & Bound)**. Se ha optimizado la actualización de la cota inferior de forma incremental en cada paso de la recursión, evitando el recálculo total.

Además, se realiza una fase inicial de **Greedy Search** para obtener una cota superior (Upper Bound) de calidad desde el inicio, lo que permite podar ramas ineficientes de manera muy temprana.

3. Experimentación

Se han realizado pruebas con diferentes tamaños de N para evaluar el rendimiento. Los resultados se muestran en la siguiente tabla:

N	Tiempo (ms)	Nodos Generados	Valor Óptimo
6	0	26	42
9	0	429	73
12	0	1,697	65
15	14	44,538	74

Se observa un crecimiento exponencial en el número de nodos, coherente con la naturaleza NP-Hard del problema. No obstante, la cota inferior implementada es lo suficientemente fuerte como para manejar $N = 15$ en tiempos insignificantes.

4. Programación Lineal Entera (PLI)

Se ha desarrollado una solución alternativa utilizando el modelo de programación lineal entera. La formulación utiliza variables binarias $z_{i,j}$ que indican si los participantes i y j pertenecen al mismo equipo.

Modelo:

- **Minimizar:** $\sum_{i < j} (C_{i,j} + C_{j,i}) \cdot z_{i,j}$
- **Sujeto a:**
 - $\sum_{j \neq i} z_{i,j} = 2, \quad \forall i$ (Cada uno tiene exactamente 2 socios).
 - $z_{i,j} + z_{j,k} - 1 \leq z_{i,k}, \quad \forall i, j, k$ (Transitividad/Clicas de 3).

Este modelo se implementó en Python con la librería PuLP, validando los resultados obtenidos por el algoritmo de Ramificación y Poda.

5. Conclusiones

El algoritmo de Ramificación y Poda es altamente dependiente de la calidad de la cota inferior. Con una cota basada en los pesos mínimos del grafo, el espacio de búsqueda se reduce drásticamente. La programación lineal ofrece una forma robusta de validar la correctitud del algoritmo principal, permitiendo asegurar que las podas realizadas no eliminan soluciones óptimas.